

AI-Powered Mobility Assistant for Visually Impaired

software/main.py

```
import cv2
from yolo_detection import detect_objects
from ocr_reader import extract_text
from gps_module import get_location
from voice_output import speak

camera = cv2.VideoCapture(0)

print("AI Mobility Assistant Started...")

while True:
    ret, frame = camera.read()

    if not ret:
        break

    # Object Detection
    detected_objects = detect_objects(frame)

    for obj in detected_objects:
        label = obj['label']
        confidence = obj['confidence']

        print(f"Detected: {label} ({confidence:.2f})")

        if confidence > 0.5:
            speak(f"Detected {label}")

    # OCR Detection
    extracted_text = extract_text(frame)

    if extracted_text.strip() != "":
        print("OCR Text:", extracted_text)
        speak(extracted_text[:100])

    # GPS Location
    location = get_location()

    if location:
```

```

        print("Location:", location)

# Display Output Window
cv2.imshow("AI Mobility Assistant", frame)

key = cv2.waitKey(1)

if key == 27:
    break

camera.release()
cv2.destroyAllWindows()

```

software/yolo_detection.py

```

from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")

def detect_objects(frame):
    results = model(frame)

    detected = []

    for result in results:
        boxes = result.boxes

        for box in boxes:
            cls = int(box.cls[0])
            conf = float(box.conf[0])
            label = model.names[cls]

            x1, y1, x2, y2 = map(int, box.xyxy[0])

            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(frame,
                        f"{label} {conf:.2f}",
                        (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX,
                        0.5,
                        (0, 255, 0),
                        2)

```

```
        detected.append({
            'label': label,
            'confidence': conf
        })

    return detected
```

software/ocr_reader.py

```
import pytesseract
import cv2

def extract_text(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    threshold = cv2.threshold(
        blurred,
        0,
        255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )[1]

    text = pytesseract.image_to_string(threshold)

    return text
```

software/gps_module.py

```
import serial

def get_location():
    try:
        gps = serial.Serial('/dev/ttyUSB0', baudrate=9600, timeout=1)

        data = gps.readline().decode('utf-8', errors='ignore')
```

```
        return data

    except:
        return "GPS Not Available"
```

software/voice_output.py

```
import pyttsx3

engine = pyttsx3.init()

engine.setProperty('rate', 150)
engine.setProperty('volume', 1.0)

def speak(text):
    print("Speaking:", text)

    engine.say(text)
    engine.runAndWait()
```

software/face_recognition.py

```
import face_recognition
import cv2

def recognize_face(frame, known_image_path, known_name):
    known_image = face_recognition.load_image_file(known_image_path)
    known_encoding = face_recognition.face_encodings(known_image)[0]

    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    face_locations = face_recognition.face_locations(rgb_frame)
    face_encodings = face_recognition.face_encodings(
        rgb_frame,
        face_locations
    )
```

```
for face_encoding in face_encodings:
    matches = face_recognition.compare_faces(
        [known_encoding],
        face_encoding
    )

    if True in matches:
        return known_name

return "Unknown"
```

software/utils.py

```
import cv2

def resize_frame(frame, width=640):
    height = int(frame.shape[0] * width / frame.shape[1])

    return cv2.resize(frame, (width, height))
```

software/requirements.txt

```
opencv-python
numpy
pyttsx3
pytesseract
ultralytics
imutils
pygame
face_recognition
pyserial
```

software/README.md

```
# Software Module
```

This folder contains all Python source code used in the AI-Powered Mobility Assistant project.

Modules

- main.py → Main execution pipeline
- yolo_detection.py → YOLO object detection
- ocr_reader.py → OCR text recognition
- gps_module.py → GPS location tracking
- voice_output.py → Speech synthesis
- face_recognition.py → Face recognition module

hardware/gpio-connections/gpio-mapping.txt

```
GPS TX  -> Raspberry Pi RX
GPS RX  -> Raspberry Pi TX
Camera  -> Raspberry Pi CSI Port
Speaker -> Raspberry Pi Audio Jack
Power   -> 5V Power Bank
```

hardware/gps-configuration/gps_setup.txt

```
Module: NEO-6M GPS
Baud Rate: 9600
Connection Type: UART
```

hardware/README.md

```
# Hardware Module
```

Components

- Raspberry Pi 4
- Raspberry Pi Camera Module
- NEO-6M GPS Module
- Earphones / Speaker

- Power Bank

Features

- Real-time object detection
- OCR text recognition
- GPS tracking
- Embedded AI processing
- Wearable architecture